

Documento Técnico — Vocaciones STEAM API

Versión 1.0

15 de March de 2026

2. Resumen General

El proyecto "Vocaciones STEAM API" es el backend de una plataforma orientada a descubrir y fomentar las vocaciones en áreas STEAM (Science, Technology, Engineering, Art, Mathematics) a través de un test vocacional. Sirve como el motor principal de datos, lógica de negocio y recomendaciones mediante IA. Está construido utilizando TypeScript, Node.js y el framework NestJS con una arquitectura de microservicios, TypeORM para interactuar con la base de datos PostgreSQL y Google Generative AI para las recomendaciones.

3. Arquitectura del Sistema

El sistema está estructurado como un monorepo administrado por NestJS, que contiene múltiples microservicios (api-gateway, auth, users, tests, ai, mail) y una biblioteca compartida (libs/common). El API Gateway (puerto 3000) expone los endpoints HTTP RESTful hacia los clientes (PWA, web) y rutea las solicitudes a los microservicios correspondientes a través de TCP, asegurando escalabilidad e independencia. Cada microservicio maneja su propio dominio de dominio, pero la base de datos PostgreSQL se comparte a través de las entidades comunes definidas en libs.

Puertos de los servicios:

Servicio	Puerto	Protocolo
API Gateway	3000	HTTP / REST
Auth Service	3001	Microservicio TCP
Users Service	3002	Microservicio TCP
Tests Service	3003	Microservicio TCP
AI Service	3004	Microservicio TCP
Mail Service	3005	Microservicio TCP

4. Estructura de Carpetas

El repositorio sigue una convención estándar de monorepo de NestJS:

- **apps/**: Contiene las distintas aplicaciones.
- **api-gateway/**: Punto de entrada HTTP y Swagger.
- **auth/**: Microservicio para registro, login y JWT.
- **users/**: Microservicio para perfiles de usuario y configuración.
- **tests/**: Microservicio para tests vocacionales.
- **ai/**: Microservicio de IA para conectarse a Gemini.
- **mail/**: Servicio de simulación/envío de correos OTP.
- **libs/common/**: Carpeta compartida con DTOs (validaciones), entidades TypeORM, y la configuración del módulo de base de datos transversal.
- **.env**: Archivo con las variables y secretos vitales.
- **package.json**: Dependencias y scripts base.

5. Base de Datos

La API utiliza PostgreSQL con TypeORM. Las entidades principales son:

- **users**: Guarda emails, contraseñas, rol, nivel, título en el gamification.
- **user_settings**: Relacionada 1 a 1 con "users". Guarda preferencias como "darkMode", lenguaje y notificaciones.
- **otp_codes**: Almacena códigos generados para registro y recuperación de contraseñas, cuenta con tiempos de expiración.

- **vocational_tests**: Vinculado a "users" (N a 1). Guarda los puntajes STEAM y respuestas emitidas del test de un alumno.

- **ai_recommendations**: Relacionada 1 a 1 con el test. Guarda el veredicto generado por Gemini respecto a carreras y universidades.

6. Microservicios

API Gateway: Recibe y enruta solicitudes HTTP. Tiene guards JWT y la estrategia OAuth de Google. Documenta en Swagger.

Auth Service: Realiza hash de contraseñas, interconecta con Postgres (tabla "users" y "otp_codes"). Genera los JWT asimétricos o simétricos.

Users Service: Actualiza datos del perfil (tabla "users", "user_settings").

Tests Service: Analiza las respuestas, asigna calificaciones a los pilares STEAM, crea registros en "vocational_tests", e invoca remotamente al servicio de IA.

AI Service: Envía un prompt dinámico parametrizado por región y puntajes STEAM a Gemini (genAI) mediante "@google/generative-ai".

Mail Service: Captura de eventos para enviar HTML emails con Nodemailer (como los OTP de verificación).

7. Flujo de Autenticación

1. Registro: Usuario provee correo y clave. Auth crea registro inactivo y dispara el OTP por Mail Service.

2. Verificación: El usuario manda el OTP. Se comprueba con Postgres; si es válido la cuenta se activa y devuelve el JWT inicial.

3. Login Normal: Correo y clave. Si coinciden y el correo está verificado, revoca un JWT válido.

4. Google OAuth: Flujo /auth/google lleva a Google. Luego /auth/google/callback intercepta datos del perfil, crea o actualiza en bd el googleId, lo verifica y retorna a la PWA con un token.

8. JSON Web Token (JWT)

El JWT es firmado por "AuthService" empleando "JWT_SECRET" (vía config module). El payload embebe el "sub" (id), "email", y "role". El API Gateway cuenta con un "JwtStrategy" basado en Passport que intercepta el encabezado Bearer, lo verifica y decodifica volcando los datos a "req.user" si se implementa el "JwtAuthGuard".

9. Geolocalización

La variable "locationInput" se recibe durante la sumisión del test de vocación. En lugar de procesarlo nativamente, este valor se deriva como contexto geográfico al modelo de IA (Gemini) en su Prompt. De esta manera, Gemini sugiere universidades locales a proximidad en dicha Ciudad/País.

10. Variables de Entorno

Variable	Descripción	Ejemplo
PORT_GATEWAY	Puerto del gateway	3000
DB_USER/DB_PASSWORD	Credenciales Postgres	postgres
JWT_SECRET	Semilla estricta del JWT	abc123secret...
GOOGLE_CLIENT_ID	ID OAuth de GCP	x-client.apps...
GEMINI_API_KEY	Llave para Google AI	AlzaSyAXXXX

11. Cómo Levantar el Proyecto Localmente

Paso 1: Clonar el proyecto y abrir una terminal.

Paso 2: Ejecutar `npm install` para instalar dependencias.

Paso 3: Configurar el archivo `.env` garantizando accesos a la base de datos.

Paso 4: Levantar PostgreSQL y colocar crear una Base de Datos `steam_vocations`.

Paso 5: Ejecutar el archivo SQL para estructurar la DB.

Paso 6: Ejecutar los microservicios en múltiples terminales o usando `npm run start:dev [nombre-app]`:

`npm run start:dev api-gateway`

`npm run start:dev auth ...`

12. Cómo Probar la API

La plataforma principal para probar manualmente la API es Swagger. Al iniciar todos los servicios, abre un navegador en:

<http://localhost:3000/api>

Podrás inspeccionar todos los endpoints DTO. Para acceder a rutas protegidas, regístrate a través del endpoint `/auth/register`, recupera tu JWT en el `/auth/login`, y pega el string Bearer en el candado "Authorize" verde.